Dictionaries & Sets (DSA)

Day 2 • Focus: Fast lookup & unique data

🧪 Homework Answers (Day 1)

Reverse list

```
# Reverse list nums = [1, 2, 3, 4, 5] print(nums[::-1])
```

Even numbers

```
# Even numbers nums = [1,2,3,4,5,6,7,8,9,10] evens = [x for x in nums if x % 2 == 0] print(evens)
```

Merge & sort

```
# Merge & sort list1 = [3,1,5] list2 = [4,2,6] merged = sorted(list1 + list2) print(merged)
```

📖 What is a Dictionary?

Key → Value pair

Data is stored in maps of keys connecting to specific values, avoiding ordered indexes.

Fast lookup

Retrieval happens in O(1) time complexity. Extremely efficient for searching.

Like a Contact Book

You search by Name (Key) to instantly find the Phone Number (Value).

⚙️ Dictionary Example

```
contacts = { "Rahul": "9876543210", "Priya": "9123456789", "Saurav": "9000011111" } print(contacts["Rahul"])
```

✏️ Dictionary Operations

Access & Update

```
student = { "name": "Saurav", "city": "Indore", "marks": 85 } # Access values print(student["name"]) print(student.get("age", 0)) # Add or Update student["age"] = 21 student["marks"] = 90
```

Removing Items

Use `del` keyword or the `.pop()` method to remove elements safely.

```
# Delete a key-value pair del student["city"] # Pop and return the value student.pop("age")
```

🔁 Loop & Access

Iterating Items

```
for key, value in student.items():
print(key, "→", value)
```

`.items()` yields both the key and the value simultaneously.

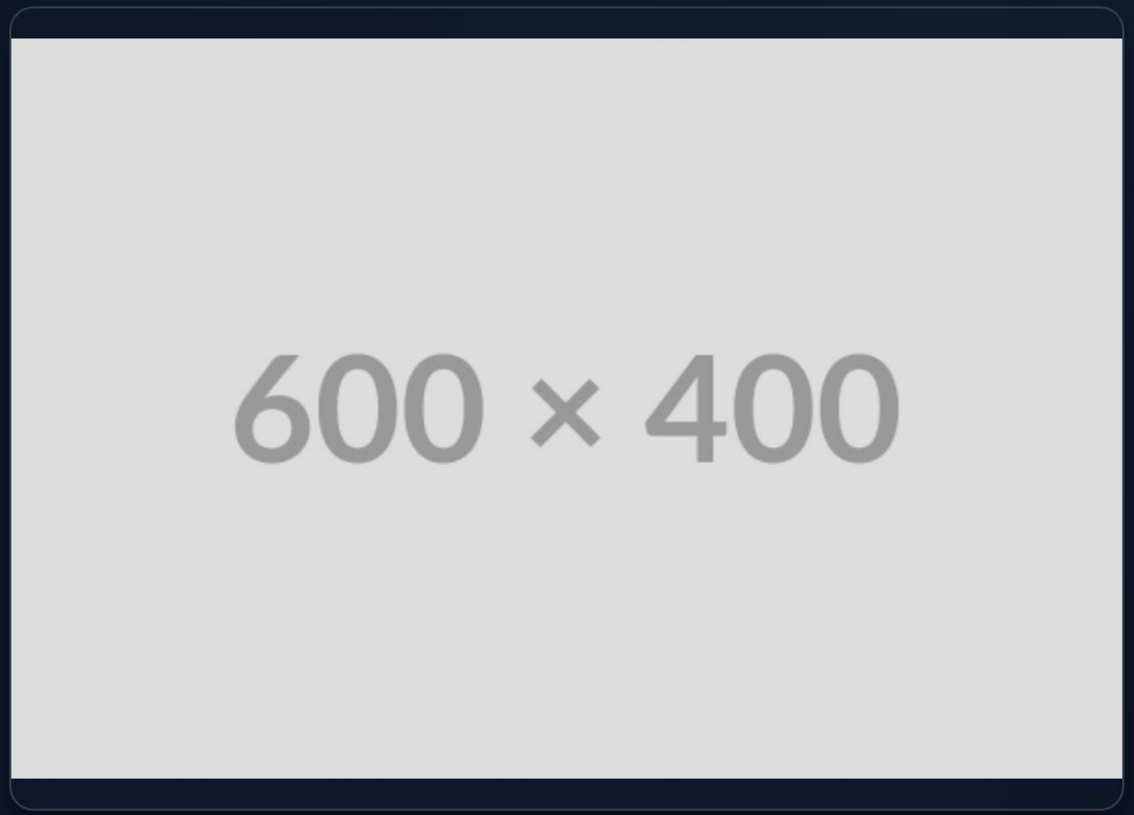
Keys, Values & In

```
print(student.keys())
print(student.values()) # Check if key exists (O(1)) print("name" in student)
print("city" in student)
```

⚡ Frequency Count (Important)

Dictionaries are the standard DSA tool for counting occurrences of items.

```
        sentence = "the cat sat on the mat the
cat" words = sentence.split() freq = {} for
word in words: freq[word] = freq.get(word, 0)
+ 1 print(freq)
```



🚀 Counter Shortcut

Python collections

```
from collections import Counter words = ["apple","banana","apple", "mango","banana","apple"] count = Counter(words) print(count) print(count.most_common(2))
```

Why use Counter?

The `Counter` class automatically builds a frequency dictionary in C, making it incredibly fast and optimized.

The `.most_common(n)` method is a built-in shortcut to retrieve the top `n` frequent elements without needing custom sorting logic.

🔵 What is a Set?

Unique Elements

Every element in a set is distinct. It serves as a mathematical set representation.

No Duplicates

Any duplicates added to a set are automatically and silently ignored.

Unordered

Sets do not maintain insertion order, and elements cannot be accessed via index.

⚙️ Set Example

Automatic Filtering

When you pass data with duplicate entries into a set, it immediately filters them down to unique elements.

```
        students =
{"Rahul","Priya","Saurav","Rahul"} # 'Rahul'
only appears once! print(students)
```

➗ Set Operations

Python Syntax

```
A = {1,2,3,4,5} B = {4,5,6,7,8} print(A | B) print(A & B) print(A - B) print(A ^ B)
```

Mathematical Equivalents

- | **(Pipe):** Union. All elements from both sets.
- & **(Ampersand):** Intersection. Only common elements.
- - **(Minus):** Difference. In A but not in B.
- ^ **(Caret):** Symmetric Difference. Elements in either A or B, but NOT both.

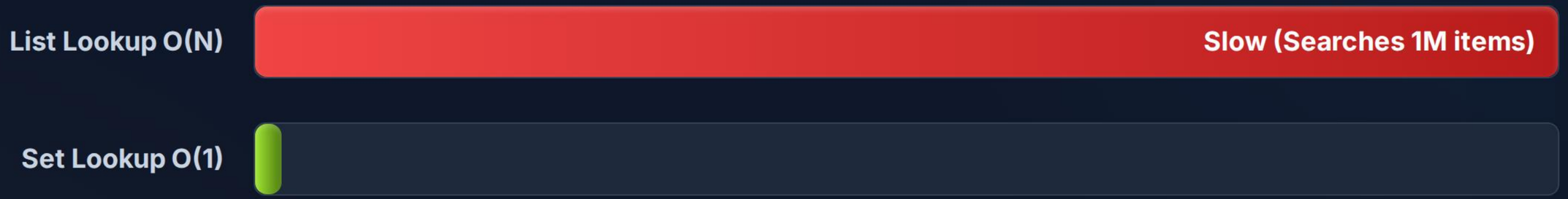
👥 Real Example

Finding Mutual Friends

Sets are perfect for resolving relationship intersections or overlaps instantly without nested loops.

```
       rahul_friends =
{"Amit","Priya","Saurav","Neha"} priya_friends
= {"Saurav","Neha","Ravi","Pooja"} common =
rahul_friends & priya_friends print(common)
```

⚡ Performance (Important)




```
# Searching in a massive List vs Set

    big_list = list(range(1000000))
print(999999 in big_list) # O(N) time


    big_set = set(range(1000000))
print(999999 in big_set) # O(1) instantaneous time
```

📊 When to Use What

Data Structure	Ordered?	Duplicates?	Primary Use Case
List `[ ]`	✓ Yes	✓ Yes	Storing sequences, preserving order
Dictionary `{k: v}`	✓ Yes (Python 3.7+)	✗ Keys: No	Key-Value mapping, fast lookups
Set `{ }`	✗ No	✗ No	Unique elements, fast math operations

🧪 Practice Problems

- **Remove Duplicates:** Given a list of repeating numbers, return a new list containing only unique numbers.
- **Most Frequent Character:** Write a script that finds the character that appears the most times in a given string.
- **Common Elements:** Given two large arrays, efficiently find all the elements that are present in both arrays.

📝 Summary

Core Concepts

- **Dictionary:** Provides O(1) lookup speeds mapped by keys.
- **Set:** Stores unique elements, completely discarding any duplicates.

Advanced Features

- **Counter:** The ultimate shortcut for frequency tracking.
- **Intersection:** The fastest way to find common elements mathematically.

🔥 Next Up

Day 3 → Loops & Functions

Image Sources

https://png.pngtree.com/thumb_back/fw800/background/20251126/pngtree-an-abstract-representation-of-digital-encryption-and-data-access-featuring-a-image_20613894.webp

Source: pngtree.com

https://i.ytimg.com/vi/UIMJLXNijXY/maxresdefault.jpg

Source: www.youtube.com

https://png.pngtree.com/png-vector/20251119/ourlarge/pngtree-glowing-neon-circles-overlap-in-a-translucent-trifecta-png-image_18006819.webp

Source: pngtree.com